DRAM Pipeline Usage Guide

A hands-on, copy-paste tutorial for dram_localization_pipeline.py

This guide is about *running the code*, not how it works internally. For the conceptual/internals walkthrough, see dram_localization_pipeline_guide.pdf instead.

AFB project · dram_localization_pipeline.py

1. Getting Started

1.1 Activate the environment

Everything this project needs (KLayout, OpenCV, NumPy, PyTorch, LightGlue, pytest) lives in the `royl` conda environment. Activate it once per terminal session:

```
conda activate royl
```

Your prompt should now start with `(royl)`. Then move into the project folder:

```
cd ~/Desktop/AFB
```

1.2 Two ways to run Python code

This trips people up constantly, so it gets its own section: your terminal has **two different prompts**, and code written for one doesn't work typed into the other.

Prompt looks like...	You're in...	You can type...
(royl) user@host:~/Desktop/AFB$	bash (the shell)	Shell commands: cd, ls, python, pip install ...
>>>	the Python interpreter	Python code: from x import y, x = 1+1, print(...) ...

Watch out: Typing a Python line like `from dram_localization_pipeline import generate_sample` directly at the `$` bash prompt fails with something like `Command 'from' not found` — bash tried to run `from` as a program. You have to start Python first.

Option A: interactive Python shell (best for exploring)

```
(royl) $ python
Python 3.12.13 ...
>>> from dram_localization_pipeline import generate_sample
>>> sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen")
>>> sample.reference_img.shape
(100, 100)
```

Everything you define (like `sample`) stays available for your next command, until you exit (`exit()` or Ctrl+D).

Option B: a one-off command from bash (no shell switch)

```
python -c "from dram_localization_pipeline import generate_sample; s = generate_sample(seed=1, tmp_dir=
```

Option C: a saved script (best for repeatable work)

Write your code into a `.py` file, then run it from bash:

```
# my_script.py
from dram_localization_pipeline import generate_sample
import cv2

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen")
cv2.imwrite("search.png", sample.search_img)
print("done")
```

```
python my_script.py
```

2. Your First Sample

The one function you need for dataset generation is `generate_sample`:

```
from dram_localization_pipeline import generate_sample

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen")
```

The tmp_dir argument

`tmp_dir` is scratch space for intermediate GDS/JSON files — they're written there and deleted automatically before the function returns. It must be a directory that **already exists**; `generate_sample` does not create it for you.

Watch out: If `tmp_dir` doesn't exist, you'll get `RuntimeError: Unable to open file: ... (errno=2) in Layout.write`. Fix it by creating the folder first, or use the auto-creating pattern below.

Option 1 — create the folder once, reuse it

```
import os
os.makedirs("/tmp/dram_gen", exist_ok=True)

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen")
```

(equivalently, from bash: `mkdir -p /tmp/dram_gen`)

Option 2 — auto-create and auto-clean a temp folder

```
import tempfile
from dram_localization_pipeline import generate_sample

with tempfile.TemporaryDirectory() as tmp:
    sample = generate_sample(seed=1, tmp_dir=tmp)
# folder is created on entry, deleted on exit -- no manual cleanup needed
```

Tip: This is what every example script in this guide uses internally. Prefer it over pointing `tmp_dir` at a real folder like your Desktop — it's not wrong to do that (generate_sample cleans up its own files either way), just messier.

Watch out: Don't run two `generate_sample(...)` calls with the **same** `tmp_dir` and the same `seed` **at the same time** from two different terminals/processes — they'll both try to write and delete the same intermediate filename (`pair_000001.gds`) and can race each other, causing a spurious "file not found" error. Simply re-running usually fixes it; using separate tmp_dir folders per process avoids it entirely.

What you get back

`generate_sample` returns a `ReferenceSearchSample` object:

Field	What it is
`sample.reference_img`	uint8 grayscale NumPy array, 100x100
`sample.search_img`	uint8 grayscale NumPy array, 1000x1000
`sample.true_center_px`	(cx, cy) — where the reference patch sits inside search_img
`sample.zoom_ratio`	1.0 by default
`sample.seed`	the seed you passed in
`sample.defect_bbox_px`	bbox of a defect overlapping the reference patch, or None

`sample.training_defect_center_px / _bbox_px`	see Section 4 — only set with single_training_defect=True

3. Saving & Viewing Images

`sample.reference_img` and `sample.search_img` are just NumPy arrays in memory — nothing is saved to disk until you explicitly write them.

Save with OpenCV

```
import cv2

cv2.imwrite("/home/nihal/Desktop/AFB/reference.png", sample.reference_img)
cv2.imwrite("/home/nihal/Desktop/AFB/search.png", sample.search_img)
```

Tip: Use a full path starting with `/` (like above) so you always know exactly where the file landed. A relative path like `"search.png"` saves into whatever directory you launched Python from — check that directory anytime with `import os; os.getcwd()`.

View the saved files

From a normal terminal (not the Python prompt)

```
xdg-open /home/nihal/Desktop/AFB/search.png
xdg-open /home/nihal/Desktop/AFB/reference.png
```

Or just double-click the files in your file manager.

Without leaving Python

```
cv2.imshow("search", sample.search_img)
cv2.imshow("reference", sample.reference_img)
cv2.waitKey(0)        # press any key in the image window to close it
cv2.destroyAllWindows()
```

4. Parameter Cookbook

Every example below is a complete, runnable `generate_sample(...)` call. All parameters are keyword arguments — mix and match freely.

4.1 Different seeds

```
for s in [1, 2, 3, 4, 5]:
    sample = generate_sample(seed=s, tmp_dir="/tmp/dram_gen")
    cv2.imwrite(f"/tmp/search_seed{s}.png", sample.search_img)
```

Same seed always reproduces the exact same output (fully deterministic — the renderer currently has no noise/blur, see the main guide's Section 3 for details).

4.2 A single, bigger or smaller array

```
sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                         rows=64, cols=85)   # default -- square, fills the canvas
```

Watch out: If you change `rows`/`cols` away from the default, the array's physical aspect ratio may no longer be square, leaving a flat empty margin on one edge of `search_img`. That's expected behavior, not a bug.

4.3 A multi-region die (multiple blocks)

```
sample = generate_sample(
    seed=1, tmp_dir="/tmp/dram_gen",
    die_block_rows=5, die_block_cols=5,        # 5x5 grid of blocks
    die_block_width_nm=5120.0,                 # each block's width (nm)
    die_block_height_nm=5120.0,                # each block's height (nm)
    die_street_width_nm=100.0,                 # gap between blocks (nm)
    die_block_variation_frac=0.1,              # +/-10% per-block randomness
)
```

Square blocks (`width == height`) only make the *whole grid* square when `die_block_rows == die_block_cols`. For a rectangular grid (e.g. 2 rows × 4 cols), solve for the correct block width first:

```
from dram_localization_pipeline import square_die_block_width_nm

rows, cols, street, block_h = 2, 4, 100.0, 5120.0
block_w = square_die_block_width_nm(rows, cols, street, block_h)

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                         die_block_rows=rows, die_block_cols=cols,
                         die_block_width_nm=block_w, die_block_height_nm=block_h,
                         die_street_width_nm=street)
```

Parameter	Effect of increasing it
`die_block_rows / die_block_cols`	More blocks in the grid (each renders smaller)
`die_block_width_nm / _height_nm`	Bigger physical block -> fewer blocks fit / each renders larger
`die_street_width_nm`	Wider visible gap between blocks
`die_block_variation_frac`	More random difference between blocks (0.0 = all identical)

4.4 Single guaranteed training defect

```
sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                         single_training_defect=True)

sample.training_defect_center_px   # (cx, cy) in search-image pixels
sample.training_defect_bbox_px     # (x0, y0, x1, y1) in search-image pixels
sample.training_defect_tile        # (row, col), each 0-9
```

Places exactly one small diagonal defect and suppresses every other random defect type (particles, scratches, CMP dishing, missing features, broken lines). Combine with a multi-region die freely:

```
sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                         die_block_rows=4, die_block_cols=4,
                         single_training_defect=True)
```

5. Common Errors & Fixes

Command 'from' not found

Cause: you typed Python code at the bash `$` prompt instead of the Python `>>>` prompt. **Fix:** run `python` first (see Section 1.2).

RuntimeError: Unable to open file: ... (errno=2) in Layout.write

Cause: `tmp_dir` doesn't exist. **Fix:** create it first (`os.makedirs(tmp_dir, exist_ok=True)`) or use `tempfile.TemporaryDirectory()` (Section 2).

RuntimeError: Unable to open file: ... (errno=2) in Layout.read

Cause: almost always a race between two processes writing to the same `tmp_dir` + `seed` combination at the same time (see the warning in Section 2). **Fix:** re-run it, or use a dedicated `tmp_dir` per process/script.

ModuleNotFoundError: No module named 'dram_localization_pipeline'

Cause: you're not in (or Python can't find) the AFB project folder. **Fix:** `cd ~/Desktop/AFB` before launching Python, or add it to `sys.path`: `import sys; sys.path.insert(0, "/home/nihal/Desktop/AFB")`.

NameError: name 'cv2' is not defined

Cause: forgot `import cv2` before calling `cv2.imwrite(...)`. **Fix:** add the import at the top of your session/script.

6. Full Worked Example

A complete, copy-paste script: generate a multi-region sample with a training defect, save both images, and print the ground truth.

```
import tempfile
import cv2
from dram_localization_pipeline import generate_sample

with tempfile.TemporaryDirectory() as tmp:
    sample = generate_sample(
        seed=7, tmp_dir=tmp,
        die_block_rows=4, die_block_cols=4,
        single_training_defect=True,
    )

out_dir = "/home/nihal/Desktop/AFB"
cv2.imwrite(f"{out_dir}/my_search.png", sample.search_img)
cv2.imwrite(f"{out_dir}/my_reference.png", sample.reference_img)

print("search_img shape:   ", sample.search_img.shape)
print("reference_img shape:", sample.reference_img.shape)
print("training defect center (px):", sample.training_defect_center_px)
print("training defect tile (row, col):", sample.training_defect_tile)

python my_script.py
```

7. Where To Go Next

For how the pipeline works internally — the GDS/SEM/DRAM concepts, the full `DRAMParams` field reference, and the matching/localization benchmark — see `dram_localization_pipeline_guide.pdf` in the same folder.